

Basi di Dati

Nicolò Giuliani

Indice

1	Introduzione	2
1.1	Sistema informativo e sistema informatico	2
1.2	Informazione e dati	2
1.3	DBMS	2
1.4	Transazioni	3
1.5	Evoluzione storica	4
1.6	DBMS e file system	4
1.7	Modelli dei dati	4
1.8	Architettura di un DBMS	5
1.9	Linguaggi per basi di dati	6
1.10	Attori e utenti	6
1.11	Pregi e difetti dei DBMS	7

1 Introduzione

1.1 Sistema informativo e sistema informatico

Base di dati (significato generico): insieme organizzato di dati che supporta lo svolgimento delle attività specifiche di un ente (istituzione, azienda, ufficio, persona).

Base di dati (significato specifico): insieme di dati gestito da un DBMS.

Sistema informativo

componente di un ente che gestisce le informazioni di interesse. Ogni ente ha il proprio sistema informativo, anche se non esplicitamente identificabile nella sua organizzazione; va studiato nel suo contesto operativo. È un concetto *indipendente* dall'automazione: esistono da secoli organizzazioni la cui unica missione è gestire dati (anagrafi, banche).

Sistema informatico

la porzione automatizzata del sistema informativo, cioè la componente che gestisce le informazioni con tecnologie informatiche.

Gerarchia di inclusione: Azienda $\supset$ Organizzazione $\supset$ Sistema informativo $\supset$ Sistema informatico.

Gestione dell'informazione: raccolta/acquisizione, archiviazione/conservazione, elaborazione/trasformazione/produzione, distribuzione/comunicazione/scambio.

1.2 Informazione e dati

Le attività umane rappresentano l'informazione in molti modi (idee informali, linguaggio naturale, disegni e diagrammi, numeri e codici) su supporti diversi (mente umana, carta, dispositivi elettronici). Nei sistemi informatici invece l'informazione è rappresentata in modo povero ma essenziale: come **dati**.

- *informazione*: fatti forniti o appresi su qualcosa o qualcuno.
- *dato*: elemento di informazione, punto di partenza fissato di un procedimento.

Il dato è il risultato del processo di organizzazione, codifica e memorizzazione dell'informazione. **Il dato da solo è inutile senza la sua interpretazione** (es. i numeri su un cartello stradale non dicono nulla se non si sa che sono orari, e quale orario vale in quale giorno).

Perché i dati? È difficile rappresentare con precisione informazione e conoscenza ricche. I dati sono una *risorsa strategica* perché sono più stabili delle altre componenti (processi aziendali, tecnologie, ruoli): es. i dati di banche e anagrafi.

1.3 DBMS

DBMS (DataBase Management System): sistema che gestisce collezioni di dati

- **grandi**
- **persistenti**
- **condivise**

garantendo

- **privatezza**
- **affidabilità**
- **efficienza**
- **efficacia**

Esempi: DB2, Oracle, SQLServer, MySQL, PostgreSQL, Access, BigQuery, SQLite.

Grandi dimensioni molto maggiori della memoria centrale; il solo limite considerato è quello fisico dei dispositivi. Ordini di grandezza: 1–5 TB (dati transazionali), 30–50 TB (dati decisionali), 500–800 TB (dati scientifici), 10^{11} record.

Persistenti il loro tempo di vita è indipendente da quello dei processi che li usano.

Condivise ogni (grande) azienda è divisa in settori, ognuno con un proprio sotto-sistema informativo. Senza condivisione nascono **ridondanza** (stessi dati ripetuti) e quindi possibile **incoerenza** (descrizioni diverse non coincidono). La base di dati è una risorsa *integrata e condivisa*, da cui la necessità di **meccanismi di autorizzazione** (attività diverse sugli stessi dati) e **controllo della concorrenza** (più utenti sugli stessi dati).

Privatezza si definiscono autorizzazioni del tipo “l’utente A può leggere tutti i dati e modificare il dato X”.

Affidabilità resistenza a malfunzionamenti hardware e software. La base di dati è una risorsa preziosa e va conservata a lungo termine. Tecnica fondamentale: la gestione delle **transazioni**.

Efficienza uso proficuo delle risorse di memoria (primaria e secondaria) e di tempo (esecuzione e risposta). Poiché i DBMS offrono moltissime funzioni, potrebbero risultare inefficienti: da qui grandi investimenti e concorrenza fra prodotti. L’efficienza è anche frutto della qualità del software.

Efficacia rendere produttive le attività degli utenti, offrendo funzionalità potenti e flessibili.

1.4 Transazioni

Una **transazione** è un insieme di operazioni non separabili (*atomiche*), corrette anche in presenza di concorrenza, con effetti definitivi.

Atomicità

o si esegue tutta la sequenza o non se ne esegue nulla. Es.: “sposta denaro dal conto A al conto B: o si preleva da A e si versa su B, o non si fa nulla”.

Concorrenza

transazioni concorrenti devono restare coerenti: due assegni sullo stesso conto incassati contemporaneamente non devono perdersi; due agenzie non devono prenotare due volte lo stesso posto sul treno.

Durabilità

il **commit** garantisce che il risultato finale sia registrato e conservato, anche in caso di concorrenza o guasti.

Due punti di vista sulle transazioni:

- *utente*: un programma che realizza una funzione utile, frequente e predefinita, con poche eccezioni previste (versamento in conto, emissione di un certificato di nascita, iscrizione all'anagrafe, prenotazione di un volo). Sono scritte in un linguaggio ospite, noto o *ad hoc*.
- *sistema*: sequenza indivisibile di operazioni (cfr. affidabilità).

1.5 Evoluzione storica

Anni '70 — nessun DBMS

le applicazioni accedono direttamente ai file del sistema operativo; i dati comuni vengono duplicati nelle applicazioni.

Anni '80 — primi DBMS

fra le applicazioni (Pascal, C) e i file si interpone il DBMS, che offre tabelle.

Anni '90 — DBMS attivi

il comportamento procedurale comune viene condiviso fra le applicazioni. Nascono le **stored procedures**, che però non sono standardizzate e soffrono di *impedance mismatch* con il linguaggio usato per esprimerle; si introducono quindi i **trigger**, regole che modellano il comportamento procedurale condiviso e sono gestite dal DBMS stesso.

Anni 2000

architettura client/server su Internet: front-end (Javascript) sul client, Web Application Server e back-end (Java), database attivo con tabelle e trigger sul server.

Anni 2010

app mobili: front-end in Java, Objective-C, HTML5; il resto dell'architettura resta invariato.

1.6 DBMS e file system

Anche un file system può gestire grandi collezioni di dati, ma offre un accesso a grana grossa: “tutto o niente”. I DBMS estendono le funzionalità del file system, offrendo più servizi in modo integrato.

Descrizione dei dati ogni programma che non usa un DBMS contiene al proprio interno la descrizione della struttura dei file che tratta: programmi diversi possono avere descrizioni diverse, generando incoerenza fra le rappresentazioni. Nei DBMS una porzione della base di dati (il **catalogo** o *dizionario*) contiene una descrizione centralizzata dei dati, condivisa da tutte le applicazioni.

1.7 Modelli dei dati

Nei DBMS i dati sono descritti a diversi **livelli di astrazione**: i programmi usano una rappresentazione di alto livello indipendente da quella di livello più basso, così una modifica al livello basso non richiede di modificare i programmi. Questo è ottenuto attraverso il concetto di **modello dei dati**.

Modello dei dati: insieme di costrutti utilizzati per organizzare i dati e descriverne il comportamento. Componente fondamentale: dare struttura ai dati tramite **costruttori di tipo**. Es.: il **modello relazionale** fornisce il costruttore *relazione*, che permette di definire insiemi di record omogenei.

Schema e istanza in ogni base di dati esistono

- lo **schema**: invariante nel tempo, descrive la struttura dei dati (aspetto *intensionale*) — es. l'intestazione delle tabelle;
- l'**istanza**: i valori effettivi, che possono cambiare rapidamente (aspetto *estensionale*) — es. il corpo delle tabelle.

Due tipologie di modelli

Modelli logici

usati dai DBMS per organizzare e memorizzare i dati; usati dai programmi, indipendenti dalla rappresentazione fisica. Es.: relazionale, reticolare, gerarchico, a oggetti, XML.

Modelli concettuali

rappresentano i dati indipendentemente da ogni sistema specifico; descrivono concetti del mondo reale e si usano nella fase preliminare di progettazione. Il più diffuso è **Entity-Relationship**.

1.8 Architettura di un DBMS

Architettura semplificata (a due livelli): utente $\rightarrow$ *schema logico* $\rightarrow$ *schema interno* $\rightarrow$ DB.

Schema logico

descrive la struttura della base di dati tramite il modello logico dei dati (es. la struttura delle tabelle).

Schema interno (o fisico)

rappresenta lo schema logico a livello di memorizzazione, con strutture dati *raw* (file, record, puntatori).

Poiché la rappresentazione logica è indipendente da quella fisica — una tabella si usa “così com'è”, indipendentemente dalla rappresentazione sottostante, che può anche variare nel tempo — nel corso si considera solo il livello logico.

Architettura standard ANSI/SPARC a tre livelli aggiunge sopra lo schema logico uno o più **schemi esterni**, ciascuno usato da un gruppo di utenti:

- **schema logico**: struttura della base di dati secondo il modello logico “principale”;
- **schema interno**: rappresentazione a livello di memorizzazione;
- **schema esterno**: descrive una parte della base di dati in un modello logico (*viste* parziali, tabelle derivate, anche da modelli diversi).

Indipendenza dei dati è conseguenza della stratificazione; l'accesso ai dati avviene sempre a livello esterno (che può coincidere con quello logico).

Indipendenza fisica

livello logico ed esterno sono indipendenti dal livello fisico: una relazione è usata allo stesso modo qualunque sia la sua realizzazione fisica, che può cambiare senza modificare i programmi.

Indipendenza logica

il livello esterno è indipendente da quello logico: si può operare su una vista senza modificare il livello logico, e modifiche “trasparenti” allo schema logico lasciano inalterato lo schema esterno.

1.9 Linguaggi per basi di dati

Un altro aspetto dell'efficacia è la disponibilità di linguaggi e interfacce diverse:

- linguaggi testuali *interattivi* (**SQL**);
- comandi SQL immersi in un linguaggio **ospite** (Pascal, Java, C);
- comandi SQL immersi in un linguaggio *ad hoc* con altre funzionalità (grafici, stampa di tabelle) — es. Oracle PL/SQL;
- interfacce amichevoli, senza linguaggio testuale.

Esempio SQL interattivo date le tabelle `Corsi(Corso, Docente, Aula)` e `Aule(Nome, Edificio, Piano)`, per ottenere i corsi che si tengono al piano terra:

```
SELECT Corso, Aula, Piano
FROM Aule, Corsi
WHERE Nome = Aula
      AND Piano = 'Terra'
```

Separazione fra dati e programmi

DML

data manipulation language: interrogazione e aggiornamento delle *istanze* della base di dati.

DDL

data definition language: definizione degli *schemi* (logico, esterno, fisico) e altre operazioni. Es.:

```
CREATE TABLE orario (
  corso      CHAR(20),
  docente    CHAR(20),
  aula       CHAR(4),
  ora        CHAR(5) )
```

1.10 Attori e utenti

- progettisti e realizzatori di DBMS;
- progettisti della base di dati e amministratori (**DBA**);
- progettisti e programmatori di applicazioni;
- utenti:
 - *utenti finali* (terminalisti): eseguono operazioni predefinite (transazioni);
 - *utenti casuali*: eseguono operazioni non previste a priori, usando linguaggi interattivi.

DBA persona o gruppo responsabile del controllo e della gestione centralizzata della base di dati (efficienza, affidabilità, autorizzazioni). Spesso il DBA si occupa anche della progettazione, tranne che nei progetti complessi.

1.11 Pregi e difetti dei DBMS

Pregi

- dati come risorsa comune, base di dati come modello della realtà;
- gestione dei dati centralizzata, standardizzabile ed economia di scala;
- disponibilità di servizi integrati;
- riduzione di ridondanze e incoerenze;
- indipendenza dei dati (favorisce sviluppo e manutenzione delle applicazioni).

Difetti

- costo dei prodotti e della loro adozione;
- funzionalità non separabili (con conseguente riduzione di efficienza).